

STUDENT HANDOUT

Food Delivery App Final Project Documentation

Build a Flutter food-delivery client that follows the Figma UI, talks to the Fake Restaurant API, and uses the same Clean Architecture you practiced in the course project.

Item	Value
Course	Flutter
Project type	Final / capstone project
UI source	Food Delivery App UI Kit (Figma Community)
Backend	Fake Restaurant API (REST / JSON)
Architecture	Clean Architecture + Cubit + GetIt (same as course project)
Base URL	https://fakerestaurantapi.runasp.net

1. Project overview

You will build a mobile food-delivery app in Flutter. Users register and sign in, browse restaurants and dishes, open a restaurant menu, add items to a cart, place an order, then review and manage their order history. The visual design comes from the Figma UI kit. All real data comes from the Fake Restaurant API.

Official links (open these first)

Resource	URL
Figma design	https://www.figma.com/design/1XfcAbRTl0PSyTANsOI0gY/Food-Delivery-App-UI-Kit-Food-App-Design-Food-Mobile-App-Delivery-UI--Community-?node-id=47-23
API docs	https://fakerestantapi.runasp.net/Docs.html
API GitHub (optional)	Linked from the API docs site

What “done” looks like

- A runnable Flutter app (Android and/or iOS emulator is enough).
- Clean Architecture folders like the course project: **core** + feature packages.
- Remote calls isolated in the data layer (Dio). UI never calls the API directly.
- Cubit (or Bloc) for each feature, registered with GetIt.
- Loading, empty, and error states on every network screen.
- Session persistence: after a successful login, the user stays signed in.
- A cart that can hold items from **one restaurant at a time**, then place a real API order.

2. Clean Architecture (same as the course project)

Copy the structure you used in the course app. The difference is the data source: instead of Firebase Auth / Firestore you will use REST with Dio. Keep the same separation of layers so the UI does not know how JSON or HTTP works.

Layers

Layer	Folder	Responsibility	Course equivalent
Presentation	features/<name>/presentation	Screens, widgets, Cubit, UI state	AuthCubit, LoginScreen, widgets
Domain	features/<name>/domain	Entities + abstract repository. No Flutter, no Dio.	UserEntity, AuthRepository
Data	features/<name>/data	DTOs/models, remote service, repository implementation	UserModel, FirebaseAuthService, AuthRepoImpl
Core	lib/core	DI, routing, theme, helpers, networking, shared widgets	get_it, AppRouter, AppColors, FirebaseResult

Required folder map

```
lib/  
core/  
constants/ // base URL, keys, default error text  
di/ // GetIt registrations  
helpers/ // extensions, session (save usercode)
```

```

networking/ // Dio factory, interceptors, ApiResult, safeApiCall
routing/ // routes + AppRouter
themes/ // colors, text styles from Figma
widgets/ // buttons, text fields, loaders
features/
auth/ // register, login, session, logout
restaurants/ // list, filter, details
menu/ // restaurant menu + item search
cart/ // local cart (no cart API exists)
orders/ // place order, history, details, cancel
profile/ // email, change password, delete account

```

Dependency direction (do not break this)

- Presentation → Domain (Cubit talks to the **abstract** repository).
- Data → Domain (models map to entities; impl implements the abstract repo).
- Domain depends on **nothing** from Flutter, Dio, or JSON annotations if you can avoid it.
- Core networking is shared. Feature services use the shared Dio client.

Suggested packages (stay close to the course stack)

Package	Why
flutter_bloc / bloc	Cubit + states, same as AuthCubit
get_it	Dependency injection, same as setupGetIt()
freezed + freezed_annotation	Union states (loading / success / failure)
json_annotation + json_serializable + build_runner	DTO fromJson / toJson
dio + pretty_dio_logger	HTTP client + request logs
shared_preferences or flutter_secure_storage	Persist usercode after login
cached_network_image	Menu item photos from imageUrl
equatable (optional)	Value equality for entities

Replace Firebase pieces with HTTP pieces

FirebaseResult → ApiResult<T> with success/failure. safeFirebaseCall → safeApiCall that catches DioException and maps it to an error model. FirebaseAuthService → AuthRemoteService using Dio. getCurrentUser() → read the stored usercode + email from local session.

3. Domain models you must implement

Create entities in domain and JSON models in data. Field names below match the API. Prices are decimal numbers (treat them as INR in the UI).

Entity	Fields	Notes
Restaurant	restaurantID, restaurantName, address, type, parkingLot	type is the cuisine / category
MenuItem	itemID, itemName, itemDescription, itemPrice, restaurantName, restaurantID, imageUrl	Encode spaces in imageUrl before loading
UserSession	userEmail, usercode (API key)	Password must never be stored after login

Entity	Fields	Notes
CartLine	itemName, quantity, itemPrice, restaurantID, imageUrl?	Local only. Place-order body needs itemName + quantity
MasterOrder	masterID, userID, usercode, restaurantID, grandtotal	History list item
OrderLine	orderID, userID, itemName, quantity, itemPrice, totalPrice, masterID	Details of one master order

4. Screens — what to build and what each screen does

Implement these screens in the Figma style. Each row tells you the user goal, the API (if any), and the expected behavior. Screens marked **Required** are graded. Screens marked **UI only** may exist in Figma but have no matching backend — build them only if you have time, with local mock data.

4.1 Authentication & session

Screen	What the student builds	API / data	Grade
Splash	Short branded splash. If a usercode is saved, go to Home. Otherwise go to Login (or Onboarding once).	Local session only	Required
Onboarding (1–3 pages)	Optional pager that matches Figma. Skip / Get started → Login.	None	Recommended
Sign up	Email + password (+ confirm password in UI). Validate empty fields like the course AuthCubit. On success save usercode and open Home.	POST /api/User/register	Required
Login	Email + password. On success save usercode + email. Show API error if credentials are wrong.	GET /api/User/getusercode	Required

4.2 Home, search, and discovery

Screen	What the student builds	API / data	Grade
Home	Header (greeting + location can be static or from restaurant.address). Horizontal cuisine chips built from unique restaurant.type values. Vertical list/grid of restaurants (name, type, address, parking badge). Pull to refresh. Bottom navigation: Home, Search, Orders, Profile (Cart can be an icon on Home).	GET /api/Restaurant	Required
Category / filter	Tapping a cuisine chip filters the list. Clearing the chip shows all restaurants.	GET /api/Restaurant?category={type}	Required
Search	One search field. Search restaurants by name and/or address. Also search dishes by item name and show which restaurant they belong to. Tapping a dish opens that restaurant menu (or item sheet).	GET /api/Restaurant?name=&address= and GET /api/Restaurant/items?itemName=	Required
Sort dishes	On Search or Menu, allow sort by price low→high / high→low.	sortByprice=asc desc	Required

4.3 Restaurant, menu, and item

Screen	What the student builds	API / data	Grade
Restaurant details	Hero header with name, cuisine (type), full address, parkingLot yes/no. Then the menu list.	GET /api/Restaurant/{id} + GET /api/Restaurant/{id}/menu	Required
Menu list	Each row: image, itemName, short description, price, add button. Empty menu state if the restaurant has no items.	Same menu endpoint. Optional ?sortByprice=	Required
Item details	Large image, name, description, price, quantity stepper, Add to cart. If Figma has extras (size, spicy), keep them visual only — the API only stores itemName + quantity.	Item object already loaded from menu/items	Required

4.4 Cart and checkout

There is **no cart endpoint**. The cart lives in memory + optional local cache (shared preferences). When the user places the order you send the cart lines to Make Order.

Screen	What the student builds	API / data	Grade
Cart	List of lines with qty + / –, remove, per-line total, grand total. Empty cart illustration. If the user adds an item from a different restaurant, ask to replace the cart.	Local CartCubit	Required
Checkout	Summary of items + grand total. Delivery address can be a local text field (API does not store address on the order). Confirm button calls Make Order with the current restaurantID.	POST /api/Order/{restaurantId}/makeorder?apikey=	Required
Payment UI	Figma may show cards / wallets. Do not integrate a real gateway. A fake “Pay & place order” button that then calls Make Order is enough.	None (then Make Order)	UI only / optional

4.5 Orders and profile

Screen	What the student builds	API / data	Grade
Orders list	All master orders for the signed-in usercode. Show masterID, restaurantID (resolve name if you already fetched restaurants), grandtotal. Empty state for new users.	GET /api/Order?apikey=	Required
Order details	Lines: itemName, qty, itemPrice, totalPrice. Show grand total. Actions: delete one line, or cancel the whole master order (with confirm dialog).	GET /api/Order/{masterId}?apikey= DELETE line or master	Required
Order tracking	Figma tracking / map screen. The API has no status or courier. If you build it, fake steps (Placed → Preparing → Done) locally after checkout.	Local mock only	UI only / optional
Profile	Show email. Change password. Logout (clear session → Login). Optional: delete account.	PUT /api/User/{usercode} DELETE /api/User/{usercode}	Required (logout + show email). Password/delete recommended.

Suggested navigation graph

Splash → (session? Home : Login)

Login ↔ Signup → Home

Home → RestaurantDetails → ItemDetails → Cart → Checkout → OrderDetails
Home → Search → RestaurantDetails / ItemDetails
BottomNav: Home | Search | Orders | Profile
Orders → OrderDetails

5. User stories (acceptance behavior)

- **Register.** New email + password creates an account, returns a usercode, and opens Home.
- **Login.** Existing email + password returns the same usercode. Invalid credentials show a readable error, not a crash.
- **Session.** Killing and reopening the app keeps the user on Home if usercode is stored.
- **Browse.** Home shows the live restaurant list from the API (currently ~31 restaurants).
- **Filter.** Choosing “Biryani” (or any type) shows only that cuisine.
- **Open restaurant.** Details + menu load for that restaurantID. Images use imageUrl.
- **Search food.** Query “fish” finds items such as Fish Amritsari and Fish Curry.
- **Cart rules.** Qty updates the line total. Mixed restaurants are blocked or replace the cart.
- **Place order.** Checkout sends menuDTO and then shows the new master order / grandTotal.
- **History.** Orders tab lists master orders for this usercode only.
- **Cancel.** User can delete a single line or the whole master order, then the list refreshes.
- **Logout.** Clears usercode and returns to Login.

6. API guide for students

Base URL: <https://fakerestaurantapi.runasp.net>. All bodies are JSON. Send Content-Type: application/json on POST/PUT. Official examples: <https://fakerestaurantapi.runasp.net/Docs.html>

Critical: usercode is the API key

After register or getusercode you receive usercode (a GUID). Every Order URL needs ?apikey={usercode}. User update/delete uses the usercode in the path. Never hard-code another student's key.

6.1 Restaurants & menu (public, no API key)

Method	Endpoint	Query / body	Use in the app
GET	/api/Restaurant	—	Home list
GET	/api/Restaurant	category=Parsi Cuisine	Cuisine chip filter
GET	/api/Restaurant	name=Paradise Biryani&address=hyderabad	Restaurant search (address is contains / substring)
GET	/api/Restaurant/{id}	—	Restaurant header (id e.g. 4 = Bawarchi)
GET	/api/Restaurant/{id}/menu	sortByprice=asc desc	Menu of one restaurant
GET	/api/Restaurant/items	—	All dishes (optional discover section)
GET	/api/Restaurant/items	ItemName=fish	Food search
GET	/api/Restaurant/items	sortByprice=asc desc	Sort all dishes

Restaurant JSON

```
{
  "restaurantID": 4,
  "restaurantName": "Bawarchi",
  "address": "Hyderabad, RTC Cross Road, Telangana",
  "type": "Biryani",
  "parkingLot": true
}
```

Menu item JSON

```
{
  "itemID": 91,
  "itemName": "Haleem",
  "itemDescription": "Rich and creamy stew made with lentils, wheat, and slow-cooked meat.",
  "itemPrice": 220.0,
  "restaurantName": "Pista House",
  "restaurantID": 5,
  "imageUrl": "https://fakereRestaurantapi.runasp.net/images/haleem.jpg"
}
```

Cuisine values you will see include: Biryani, Mughlai, South Indian, Rajasthani, Bengali Cuisine, Gujarati, Multi-cuisine, Modern Indian, Hyderabad Cuisine, Thai, Parsi Cuisine, North Indian, Seafood, Fine Dining, Brewery, and others. Build chips from the live list — do not hard-code a stale set.

6.2 User (auth)

Method	Endpoint	Body / query	Use in the app
POST	/api/User/register	<code>{"userEmail":"...", "password":"..."}</code>	Sign up — response includes usercode
GET	/api/User/getusercode	<code>UserEmail=&Password=</code>	Login — returns { usercode }
GET	/api/User	—	Optional debug only. Do not show other users' passwords in the UI.
PUT	/api/User/{usercode}	JSON string of the new password	Change password on Profile
DELETE	/api/User/{usercode}	—	Delete account, then clear session

Register response: `{"userEmail":"...", "password":"...", "usercode":"6f3d1852-..."}`. Login response: `{"usercode":"cbc4ecf6-..."}`. Save email yourself; getusercode does not echo it.

6.3 Orders (protected — apikey required)

Method	Endpoint	Body / query	Use in the app
GET	/api/Order?apikey={usercode}	—	Orders tab (master orders)
GET	/api/Order/{masterId}?apikey={usercode}	—	Order details (line items)
POST	/api/Order/{restaurantId}/makeorder?apikey={usercode}	See body below	Checkout confirm
DELETE	/api/Order/master/{masterId}?apikey={usercode}	—	Cancel whole order
DELETE	/api/Order/{orderId}?apikey={usercode}	—	Remove one line from an order

Make-order request body

```
{
  "menuDTO": [
    { "itemName": "Kofta Curry", "quantity": 2 },
    { "itemName": "Sheer Korma", "quantity": 2 }
  ]
}
```

Response shape: `{"fullorder": [ {orderId, userID, itemName, quantity, itemPrice, totalPrice, masterID}, ... ], "grandTotal": 900 }`. A **master order** is one checkout. Each cart line becomes a single order row under the same masterID.

Master-order list item

```
{
  "masterID": 4,
  "userID": "...",
  "usercode": "...",
  "restaurantID": 1,
  "grandtotal": 2750
}
```

6.4 Endpoints you must NOT treat as real features

Endpoint	Why
POST /api/Restaurant	Docs say the object is not saved. Do not build "add restaurant" as a graded feature.
POST /api/Restaurant/{id}/additem	Same: not persisted. Do not build a merchant "add dish" flow.

6.5 Networking rules (course style)

- One Dio instance in core/networking with baseUrl and timeouts (e.g. 20s).
- PrettyDioLogger in debug only.
- Wrap every call in safeApiCall so Cubits receive ApiResult.success or failure.
- Map HTTP errors and timeouts to a short user message (like ApiConstants.defaultError).
- Encode image URLs: some files contain spaces (mutton biryani.jpg).
- itemName in makeorder must match the menu spelling exactly.
- GET /api/Restaurant/{id} returns a **single object**, not a list (the HTML docs sample is outdated).
- Do not log passwords in production builds.

7. Suggested build order

Week / step	Deliverable
1. Project skeleton	Flutter app, theme from Figma, routing, GetIt, Dio, ApiResult, empty feature folders
2. Auth	Register, login, splash session, logout — same Cubit pattern as the course
3. Restaurants	Home list, category chips, restaurant details header
4. Menu & search	Menu list + images, item details, search items, sort by price
5. Cart	Local cart, restaurant lock, qty, totals
6. Orders	Make order, history, details, delete line / delete master
7. Polish	Figma spacing, skeleton loaders, empty/error, profile password, README

8. UI implementation notes

- Extract colors and text styles from Figma into core/themes (like AppColors in the course app).
- Reuse widgets: primary button, input field, restaurant card, food tile, price text, bottom nav.
- Use CachedNetworkImage with a placeholder and error icon for every dish photo.
- Show Skeletonizer (or your own shimmer) while lists load — you already used this on splash in the course project.
- SafeArea + scroll views: Figma frames are iPhone sized; test a small Android emulator too.
- Bottom navigation should keep tab state (IndexedStack or similar).
- Currency: format itemPrice as a currency (e.g. ■220). The API does not send a currency code.

9. Grading checklist

Area	What instructors will check	Weight (guide)
Architecture	Layers, no API calls from widgets, GetIt, Cubit states	25%
Auth & session	Register, login, persist usercode, logout	15%
Restaurants & menu	Live list, filter, details, images, search, sort	20%

Area	What instructors will check	Weight (guide)
Cart & order	Local cart rules + successful makeorder + history + delete	20%
UI fidelity	Looks like the Figma kit: layout, type, colors, empty/error	15%
Code quality	Naming, error handling, no crashes, short README	5%

10. What to submit

- Git repository (or zip) of the Flutter project.
- README: how to run, test account email you created, and a 1-paragraph architecture note.
- Do not commit secrets. usercode is per user and is created at runtime.

Remember

The Figma file is the look. The API is the truth for data. Clean Architecture is the rule for code. If those three stay aligned, the project is complete.